تقرير المشروع - ثمانية للBackend

موسى ابن راشد

https://github.com/mibnrashid/ThmanyahProject

1- كيف حليت المشكلة

بنيت المشروع باستخدام NestJS مع تقسيم واضح للموديولات، عشان يكون الشغل مرتب وسهل يتطور بعدين:

- CMS Module لمسارات الإدارة الداخلية (إضافة، تعديل، حذف، وعرض داخلي).
- Discovery Module لمسارات المستخدم النهائي (استعراض البرامج والبحث).
- Programs Module كطبقة مشتركة للدومين والوصول للبيانات.

استخدمت PostgreSQL مع TypeORM، وصممت جدول programs بحيث يحتوي على :العنوان، الوصف، التصنيف، اللغة، المدة، تاريخ النشر، وتواريخ الإنشاء/التحديث.

كذلك وثّقت الـ API باستخدام Swagger على api/، واستخدمت DTO validation للتحقق من المدخلات، مع Pagination واضحة (page and limit) وبحد أعلى للنتائج.

2- أهم الصعوبات اللي واجهتني

بصراحة أكبر تحدي كان إني كنت داخل على المشروع بدون خبرة سابقة في الـbackend

أبرز الصعوبات:

- فهم المفاهيم الأساسية: API, endpoint, request/response, HTTP
- استيعاب Node.js وكيف تشتغل بنية المشاريع فيه
- مراجعة JavaScript وTypeScript بشكل عملي يخدمني بالمشروع

- فهم مفاهيم NestJS مثل modules, controllers, services, DTOs, dependency injection
- المحافظة على low coupling وحدود واضحة بين المكونات بدون تعقيد زائد

وبرضوا كان فيه تحدي في الموازنة بين السرعة والفهم. استخدمت أدوات AI بالمساعدة، لكن كنت حريص أفهم كل سطر قبل ما أعتمده.

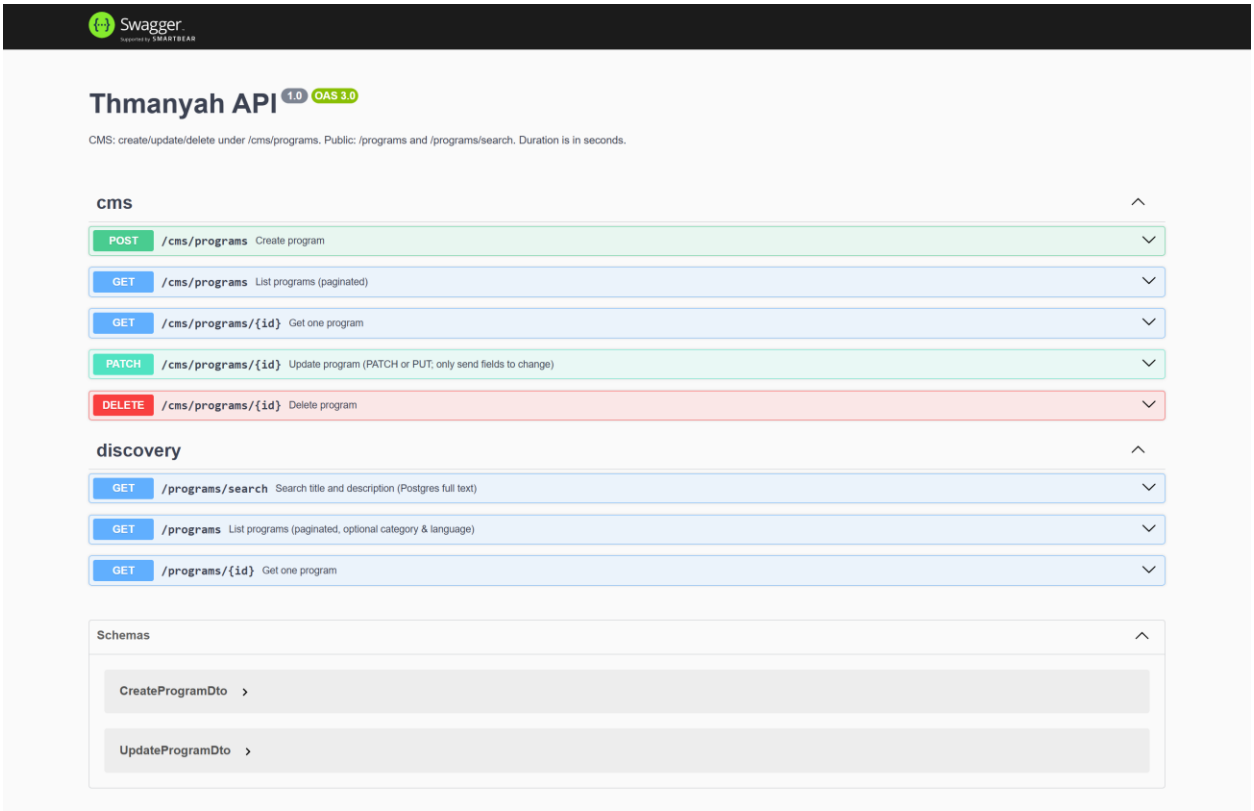
3- رحلة التعلم والجدول الزمني (5 أيام)

- **اليوم 1-2:** مشاهدة فيديوهات YouTube عن كل المفاهيم اللي احتجتها (APIs, endpoints, HTTP, Next.js, Node.js, NestJS, Express) مع مراجعة JavaScript and TypeScript
- **اليوم 3:** فهم متطلبات المشروع وتحليل التدفق المعماري وتقسيم الموديولات
- **اليوم4-5:** تنفيذ المشروع كامل + فهمه + كتابة التقرير

4- وش تم إنجازه فعليًا

- تنفيذ CMS CRUD كامل للبرامج.
- تنفيذ Discovery للاستعراض، الفلترة، والبحث.
- تفعيل التحقق من المدخلات عبر DTO and UUID pipes.
- ربط قاعدة البيانات PostgreSQL عبر TypeORM.
- تطبيق Pagination والبحث.
- توثيق المسارات عبر Swagger.

5- عرض مستند Swagger



6- تحسينات مستقبلية

لو فرضا كملت على المشروع، غالبًا التحسينات المطلوبة:

- تحسين البحث والفهرسة للبيانات الكبيرة ومتعددة اللغات.
- إضافة caching وتهيئة أفضل للتوسع تحت ضغط عالي.
- دعم استيراد البيانات من مصادر متعددة.
- إضافة auth/authorization لمسارات CMS.
- إضافة اختبارات تلقائية (unit/integration/e2e).